

Grafting is mounting is capability composition (design note)

Draft (2026-08-10, Claude, from a design conversation with Mathijs). Status: observation + design direction for the future capability layer; no decision taken.

One operad, three appearances

The mounting operad of `aips/draft/operadic-canonicity.md` (operations = prefix-free mountings; composition = grafting; depths add, cylinder masses multiply) is not specific to fast state. The same algebra appears three times in the programme:

1. **Fast state (B4)**: heads grafted into the fast-state tree under the Kraft budget — Expo 2’s model.
2. **The namespace / mount tree (convo [458]–[464], Mathijs’s proposal)**: allocations and external resources mounted at mountpoints of the virtual data space; [464] constructs the address space as the zero-padding colimit and the mount table as a finite antichain with annotated leaves. Mounting an allocation IS grafting its subtree at the mountpoint.
3. **Capabilities (convo [127]–[137])**: capabilities are *addresses of subtrees* — a cap is a prefix path, separation is prefix-incomparability, delegation/attenuation is grafting deeper. The mounting operad is the composition algebra of capabilities; mount = operad substitution.

What pricing adds

The grafting law prices namespace composition exactly: access cost to a mounted resource = (distance to the mountpoint) + (distance within the resource), with Kraft saying *near the root is scarce* — the namespace reading of “only so much fits nearby”: hot mounts crowd each other, for data exactly as for heads.

Heterogeneous backing (new ingredient, not yet in any AIP): different subtrees may carry different level-price schedules — RAM at $c(\ell) = 2^{\ell/2}$ -ish, disk and network mounts with large entry costs and their own dimensions. The namespace tree becomes a tree *decorated with cost models*, composed along grafting; the (S)-characterization (operadic-canonicity §7.1) says exactly when nested schedules compose cleanly (the grafting character must stay affine-multiplicative through the mount). This is the honest model of “map a remote resource into your namespace”: the mount’s entry cost is the latency, its internal schedule the medium’s geometry.

Payoff

When the capability calculus (B3 tier) arrives, mount needs no new cost semantics: it is operad substitution and the composition law is already a theorem. Same pre-connection move as AIP-5 §8b (credits = exchange form): the calculus should *define* its namespace/mount layer as this operad rather than invent one. Convo [129]’s unification (registers, caches, RAM, page tables, capabilities — one decorated dyadic tree with depth cost) is this note’s ancestor; the operad is that sentence made compositional.

Open design questions (for when this activates)

1. Mount-table mutation (mount/unmount at runtime) vs the static grafting algebra — is unmount an up-like potential-consuming free operation, or priced (it is eviction of a namespace entry)? Connects to cache-v0 §4b’s commit reading.

2. Heterogeneous-schedule composition: is the composite of two (S)-characters through a mount again an (S)-character on the composite tree, or only piecewise? (Expected: piecewise, and that is fine — state it.)
3. The Kraft budget for mounts: is there an entropy theorem for namespace layout (frequently-accessed mounts near the root = source coding over the mount table)? Expected: yes, verbatim — Expo 2 with “head” replaced by “mount”. Cheap first theorem when this layer activates.